Lab 1: @Controller and Thymeleaf

Spring Boot Todo List · @Controller + Thymeleaf + in-memory store

@Controller + Thymeleaf + in-memory data store.

No JPA / no MySQL this session.

What you will learn

- CRUD Todo list in the browser (Thymeleaf pages).
- Todo item fields: id (auto increment), title, description, due date, completed.
- Packages: model, dto, service, controller.
- Form binding + Bean Validation.
- Store data in an ArrayList

✓ Step 1. Create the project with Spring Initializr

Open https://start.spring.io and set:

1. Project: Maven · Language: Java · Spring Boot: 4.1.x
2. Group: com.example Artifact: todolist Package: com.example.todolist
3. Packaging: Jar Java: 25 or 26
4. Dependencies: Spring Web, Thymeleaf, Validation

Click GENERATE, unzip, open the folder in VS Code.

Why we need these three dependencies?

- Spring Web: DispatcherServlet, @Controller, form POST, @RestController.
- Thymeleaf: HTML views, server-side templates.
- Validation: use annotation for doing validation: @NotBlank, @Valid.

Confirm pom.xml contains (Initializr already wrote this):

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

src/main/resources/application.properties: turn off template cache while you edit HTML:

```
server.port=8080
spring.thymeleaf.cache=false
```

✓ Step 2. Create packages

Under com.example.todolist create four packages.

```
src/main/java/com/example/todolist/
├── TodolistApplication.java         # application boot
├── model/Todo.java                  # domain object
├── dto/TodoRequest.java             # form object
├── service/TodoService.java         # business + in-memory list
└── controller/TodoController.java   # @Controller + Thymeleaf only

src/main/resources/templates/todos/
├── list.html                        # table + links
└── form.html                        # create / edit
```

Meaning

- model: represents data in the database (or the core data structure). It contains all fields (including ID, password, created_at, etc.).
- dto (Data Transfer Object): represents data transmitted over the network (from client browser to Controller or from Controller to client browser).
  - Data security: do not expose sensitive fields (such as passwords or secret keys) to the View or API.
  - Data validation: place validation annotations (e.g., `@NotBlank`, `@FutureOrPresent`) on the DTO to capture and validate data submitted via forms or JSON, thereby avoiding cluttering the Model class.
  - Purpose-specific (tailored data): an ID does not exist during creation but is required for viewing. Sharing a single model results in unused fields being null or necessitates cumbersome handling.
- service: contains all of the application's business logic.
- controller: HTTP + HTML.

✓ Step 3. Model:Todo

model/Todo.java: this is what you keep data saved in memory.

```
package com.example.todolist.model;

import java.time.LocalDate;

public class Todo {

    private Long id;              // auto: 1, 2, 3, ...
    private String title;
    private String description;
    private LocalDate dueDate;
    private boolean completed;

    public Todo() {}

    public Todo(Long id, String title, String description,
                LocalDate dueDate, boolean completed) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.dueDate = dueDate;
        this.completed = completed;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public LocalDate getDueDate() { return dueDate; }
    public void setDueDate(LocalDate dueDate) { this.dueDate = dueDate; }

    public boolean isCompleted() { return completed; }
    public void setCompleted(boolean completed) { this.completed = completed; }
}
```

✓ Step 4. DTO:TodoRequest (form binding + validation)

dto/TodoRequest.java

```
package com.example.todolist.dto;

import com.example.todolist.model.Todo;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import org.springframework.format.annotation.DateTimeFormat;

import java.time.LocalDate;

public class TodoRequest {

    @NotBlank(message = "Title is required")
    @Size(max = 100, message = "Title must be at most 100 characters")
    private String title;

    @Size(max = 500, message = "Description must be at most 500 characters")
    private String description;

    @DateTimeFormat(iso = DateTimeFormat.ISO.DATE)
    private LocalDate dueDate;

    private boolean completed;

    public static TodoRequest from(Todo todo) {
        TodoRequest r = new TodoRequest();
        r.setTitle(todo.getTitle());
        r.setDescription(todo.getDescription());
        r.setDueDate(todo.getDueDate());
        r.setCompleted(todo.isCompleted());
        return r;
    }

    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public LocalDate getDueDate() { return dueDate; }
    public void setDueDate(LocalDate dueDate) { this.dueDate = dueDate; }

    public boolean isCompleted() { return completed; }
    public void setCompleted(boolean completed) { this.completed = completed; }
}
```

Model vs DTO

Todo.java: stored object from database (has id field).

TodoRequest.java: input from the form (no id). Never bind the HTML form directly to the stored list item.

Validation stays on the DTO so RESTful API can reuse.

✓ Step 5. Service:in-memory CRUD

service/TodoService.java: ArrayList + AtomicLong.

AtomicLong: initialize an automatic ID generator starting at 1. The `AtomicLong` type ensures that ID increments are thread-safe.

Keyword `final`: ensure that the memory addresses of these two variables `store` and `sequence` are not reassigned after initialization.

```java
package com.example.todolist.service;

import com.example.todolist.dto.TodoRequest;
import com.example.todolist.model.Todo;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class TodoService {

    private final List<Todo> store = new ArrayList<>();
    private final AtomicLong sequence = new AtomicLong(1);

    public List<Todo> findAll() {
        return new ArrayList<>(store);
    }

    public Todo findById(Long id) {
        return store.stream()
                .filter(t -> t.getId().equals(id))
                .findFirst()
                .orElseThrow(() -> new IllegalArgumentException("Todo not found: " + id));
    }

    public Todo create(TodoRequest req) {
        Todo todo = new Todo(
             sequence.getAndIncrement(),
             req.getTitle(),
             req.getDescription(),
             req.getDueDate(),
             req.isCompleted()
        );

        store.add(todo);
        return todo;
    }

    public Todo update(Long id, TodoRequest req) {
        Todo todo = findById(id);
        todo.setTitle(req.getTitle());
        todo.setDescription(req.getDescription());
        todo.setDueDate(req.getDueDate());
        todo.setCompleted(req.isCompleted());
        return todo;
    }

    public void deleteById(Long id) {
        Todo todo = findById(id);
        store.remove(todo);
    }
}
```

✓ Step 6. @Controller: HTTP + form binding

controller/TodoController.java: constructor injection. HTML forms have no DELETE, so delete is POST /todos/{id}/delete.

```
package com.example.todolist.controller;

import com.example.todolist.dto.TodoRequest;
import com.example.todolist.service.TodoService;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/todos")
public class TodoController {
    private final TodoService todoService;
    public TodoController(TodoService todoService) {
        this.todoService = todoService;
    }

    @GetMapping
    public String list(Model model) {
        model.addAttribute("todos", todoService.findAll());
        return "todos/list";
    }

    @GetMapping("/new")
    public String createForm(Model model) {
        model.addAttribute("todo", new TodoRequest());
        model.addAttribute("todoId", null);
        return "todos/form";
    }

    @PostMapping
    public String create(@Valid @ModelAttribute("todo") TodoRequest request,
                         BindingResult result,
                         Model model) {
        if (result.hasErrors()) {
            model.addAttribute("todoId", null);
            return "todos/form";
        }
        todoService.create(request);
        return "redirect:/todos";
    }
```

TodoController.java (continued)

```
    @GetMapping("/{id}/edit")
    public String editForm(@PathVariable Long id, Model model) {
        model.addAttribute("todo", TodoRequest.from(todoService.findById(id)));
        model.addAttribute("todoId", id);
        return "todos/form";
    }

    @PostMapping("/{id}")
    public String update(@PathVariable Long id,
                         @Valid @ModelAttribute("todo") TodoRequest request,
                         BindingResult result,
                         Model model) {
        if (result.hasErrors()) {
            model.addAttribute("todoId", id);
            return "todos/form";
        }
        todoService.update(id, request);
        return "redirect:/todos";
    }

    @PostMapping("/{id}/delete")
    public String delete(@PathVariable Long id) {
        todoService.deleteById(id);
        return "redirect:/todos";
    }
}
```

✓ Step 7. Todo list page

templates/todos/list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <title>Todos</title>
    <style>
        body { font-family: Segoe UI, sans-serif; margin: 24px; }
        table { border-collapse: collapse; width: 100%; }
        th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
        a, button { margin-right: 8px; }
    </style>
</head>
<body>
<h1>Todo list</h1>
<p><a th:href="@{/todos/new}">New todo</a></p>
<table>
    <thead>
    <tr>
        <th>ID</th><th>Title</th><th>Due</th><th>Done</th><th></th>
    </tr>
    </thead>
    <tbody>
    <tr th:each="t : ${todos}">
        <td th:text="${t.id}">1</td>
        <td th:text="${t.title}">title</td>
        <td th:text="${t.dueDate}">2026-01-01</td>
        <td th:text="${t.completed} ? 'Yes' : 'No'">No</td>
        <td>
            <a th:href="@{/todos/{id}/edit(id=${t.id})}">Edit</a>
            <form th:action="@{/todos/{id}/delete(id=${t.id})}"
                  method="post" style="display:inline">
                <button type="submit">Delete</button>
            </form>
        </td>
    </tr>
    </tbody>
</table>
</body>
</html>
```

- th:each: we will loop through each todo. The ${todos} variable comes from model.addAttribute in the list() method of TodoController.
- th:href / th:action: build URLs. (id=${t.id}) fills {id}.

✓ Step 8. Todo create / edit page

templates/todos/form.html:

- one template, two actions.
- todoId == null means create.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8"/>
    <title>Todo form</title>
    <style>
        body { font-family: Segoe UI, sans-serif; margin: 24px; max-width: 520px; }
        label { display: block; margin-top: 12px; font-weight: 600; }
        input[type=text], textarea, input[type=date] { width: 100%; padding: 6px; }
        .err { color: #b00020; font-size: 0.9rem; }
    </style>
</head>
<body>
<h1 th:text="${todoId == null} ? 'New todo' : 'Edit todo'">New todo</h1>

<form th:object="${todo}" method="post"
      th:action="${todoId == null} ? @{/todos} : @{/todos/{id}(id=${todoId})}">

    <label>Title</label>
    <input type="text" th:field="*{title}"/>
    <div class="err" th:errors="*{title}"></div>

    <label>Description</label>
    <textarea th:field="*{description}" rows="4"></textarea>
    <div class="err" th:errors="*{description}"></div>

    <label>Due date</label>
    <input type="date" th:field="*{dueDate}"/>

    <label>
        <input type="checkbox" th:field="*{completed}"/> Completed
    </label>

    <p>
        <button type="submit">Save</button>
        <a th:href="@{/todos}">Cancel</a>
    </p>
</form>
</body>
</html>
```

- th:object="${todo}": must match @ModelAttribute("todo").
- th:field="*{title}": name, id, and value. This is form binding.
- th:errors: shows the message from @NotBlank / @Size when BindingResult has errors.
- *{...}: property on the object. ${...} is the Spring model.

✓ Step 9. Run and check CRUD operations

Run TodolistApplication and check CRUD operations.
Open http://localhost:8080/todos page
New todo -> leave title empty -> Save. You must see “Title is required”.
Fill title + due date -> Save. Row appears in the table. id = 1.
Edit -> change completed -> Save. List shows Yes.
Delete -> row disappears.
Stop and start the app. The list is empty. In-memory store does not survive restart.

If the form does not bind

Check:
(1) th:object name = todo,
(2) getters/setters on TodoRequest,
(3) @Valid immediately followed by BindingResult,
(4) checkbox field is boolean completed / isCompleted().